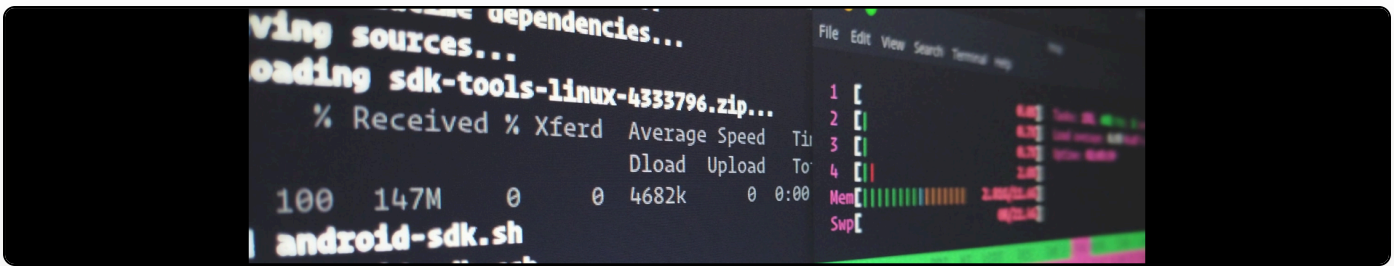
Publishing articles in PDF via XML/XSLT using Open Journal Systems 2.3.6

Cite as: Martin Paul Eve, "Publishing articles in PDF via XML/XSLT using Open Journal Systems 2.3.6", *eve.gd: Martin Paul Eve*, September 14, 2011, https://doi.org/10.59348/7hd2c-n7m52.



This is a post detailing my experiments with Open Journal Systems 2.3.6 and the current state of producing galleys for an article from a single XML file. As shall be seen in the conclusion, no currently functional plugin allows this feature. This will, therefore, be the first of several posts that will cover not only writing an OJS plugin from scratch, but also aim to fill this gap.

PDF

At present OJS contains a plugin called XMLGalley which is designed, in principle, to allow generation of article galleys, on the fly, from an XML file. There is support in this module for PDF generation and, in previous iterations, this was "working". I put scare quotes there because it introduced a different problem which can be clearly seen in the source of ArticleXMLGalleyDAO.inc.php:

```
// WARNING: The below code is disabled because of bug #5152. When a galley
// exists with the same galley_id as an entry in the article_xml_galleys table,
// editing the XML galley will corrupt the entry in the galleys table for the
// same galley_id. This has been fixed by retiring the article_xml_galleys
// table's xml_galley_id in favour of using the galley_id instead, but this
// means that only a single derived galley (=XHTML) is possible for an XML
// galley upload.
```

As is made clear from the penultimate line, the fix that has been introduced here has been to remove PDF support so that articles are generated purely from XHTML.

Now, how is this working?

The key to working out the flow in an OJS plugin is to identify the hooks it deploys. In this case XMLGalleyPlugin.inc.php provides the necessary info:

```
// NB: These hooks essentially modify/overload the existing ArticleGalleyDAO methods
HookRegistry::register('ArticleGalleyDAO::getArticleGalleys', array(&$xmlGalleyDao,
'appendXMLGalleys') );
HookRegistry::register('ArticleGalleyDAO::insertNewGalley', array(&$xmlGalleyDao,
'insertXMLGalleys') );
HookRegistry::register('ArticleGalleyDAO::deleteGalleyById', array(&$xmlGalleyDao,
'deleteXMLGalleys') );
HookRegistry::register('ArticleGalleyDAO::incrementGalleyViews', array(&$xmlGalleyDao,
'incrementXMLViews') );
HookRegistry::register('ArticleGalleyDAO::_returnGalleyFromRow', array(&$this,
'returnXMLGalley') );
HookRegistry::register('ArticleGalleyDAO::getNewGalley', array(&$this, 'getXMLGalley')
);

// This hook is required in the absence of hooks in the viewFile and download methods
HookRegistry::register( 'ArticleHandler::viewFile', array(&$this, 'viewXMLGalleyFile')
);
HookRegistry::register( 'ArticleHandler::downloadFile', array(&$this,
'viewXMLGalleyFile') );
```

The most important hooks for our purposes of understanding are insertXMLGalleys (fired when the user uploads an XML galley) and viewXMLGalleyFile (called when the user attempts to download a PDF).

Now, at present, the insertXMLGalleys function has the following code commented out, in support of bug #5152:

```
/*

                        // if we have enabled XML-PDF galley generation (plugin
setting)
                        // and are using the built-in NLM stylesheet, append a PDF
galley as well
                        $journal =& Request::getJournal();
                        $xmlGalleyPlugin =& PluginRegistry::getPlugin('generic', $this-
>parentPluginName);

                        if ($xmlGalleyPlugin->getSetting($journal->getId(), 'nlmPDF')
== 1 &&
                                $xmlGalleyPlugin->getSetting($journal->getId(),
'XSLstylesheet') == 'NLM' ) {

                                // create a PDF galley
                                $this->update(
                                        'INSERT INTO article_xml_galleys
                                                (galley_id, article_id, label,
galley_type)
                                                VALUES
                                                (?, ?, ?, ?)',
                                        array(
                                                $galleyId,
                                                $galley->getArticleId(),
                                                'PDF',
                                                'application/pdf'
                                        )
                                );

                        }*/
```

Reading this, it is clear that the way the code originally was working was to insert an entry into the article_xml_galleys with the "application/pdf" mime type in addition to the xhtml rendered markup.

As this no longer works, the aim of this project, the source of which is at my GitHub (with virtually no content there at present), will be to write a plugin which would also hook into insertXMLGalleys, run the PDF transform on the XML file, store the file in the article_files table

and add a PDF galley, at this stage, to the native article_files table.

Stay tuned for the next installment...

Featured image by rillian under a CC-BY-SA license.